MLDL Experiment 1

Aim :- Implement Linear and Logistic Regression on real-world datasets

Introduction

1. Linear Regression

Linear Regression is one of the most fundamental and widely used supervised machine learning algorithms. It is used to model the relationship between a dependent variable (target variable) and one or more independent variables (predictor variables) by fitting a straight line to the observed data.

The main objective of linear regression is to predict continuous numerical values based on input features. For example, it can be used to predict house prices based on area, salary based on experience, or temperature based on time.

Mathematical Equation

Linear regression models the relationship between a dependent variable y and one or more independent variables X by fitting a linear equation:

$$\boxed{y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \epsilon}$$

Where :

- β_0 is the intercept.
- β_n are the coefficients.
- ϵ is the error term.
- The objective is to minimize the **Mean Squared Error (MSE)**: $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Limitations of Linear Regression

1. Assumes a linear relationship between variables, so it cannot model non-linear data accurately.
2. Sensitive to outliers, which can distort the regression line.

3. Cannot be used for classification problems.
4. Multicollinearity between independent variables reduces model reliability.
5. Assumes constant variance of errors, which may not always hold true.

2. Logistic regression

Logistic Regression is also a supervised machine learning algorithm, but unlike linear regression, it is used for classification problems rather than predicting continuous values. It is used when the output variable is categorical, typically binary (for example: Yes/No, True/False, Spam/Not Spam, Disease/No Disease).

Instead of predicting exact numerical values, logistic regression predicts the probability that a given input belongs to a particular class. It uses a mathematical function called the sigmoid function (also known as the logistic function) to convert the output into a probability value between 0 and 1.

Mathematical Equation

Logistic regression predicts the probability that an instance belongs to a particular class using the **Sigmoid function**:

$$\boxed{P(y=1|X) = \sigma(z) = \frac{1}{1+e^{-z}}}$$

Where $z = \beta_0 + \beta_1 x_1 + ... + \beta_n x_n$.

The model minimizes the **Log Loss (Binary Cross-Entropy)**:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})]$$

Limitations of Logistic Regression

1. Assumes a linear decision boundary, so it struggles with complex non-linear data.
2. Not suitable for highly complex classification problems.
3. Sensitive to outliers and noisy data.
4. Requires sufficient data for accurate predictions.
5. Cannot predict continuous values, only categorical outcomes

Dataset Source

- Linear Regression: Medical Cost Personal Datasets
- Logistic Regression: Diabetes Dataset

Dataset Description

1. Linear Regression Dataset: Medical Cost Personal Data

This dataset is used to predict the insurance costs (**charges**) of individuals based on their demographic and health attributes.

Feature	Data Type	Description & Characteristics
Age	Integer	Range: 18–64 years. Older individuals generally have higher medical costs.
Sex	Categorical	Insurance contractor gender (male/female). Roughly equal distribution in the data.
BMI	Float	Body Mass Index (kg/m^2). Ideally 18.5 to 24.9. Higher BMI often correlates with higher charges.
Children	Integer	Number of children/dependents covered by health insurance (range: 0–5).
Smoker	Categorical	Whether the person smokes (yes/no). This is usually the strongest predictor of cost.
Region	Categorical	Residential area in the US (Northeast, Southeast, Southwest, Northwest).
Charges	Target	The medical cost billed to the individual. Range: ~$1,121 to ~$63,770.

2. Logistic Regression Dataset: Pima Indians Diabetes

This dataset consists of clinical diagnostic measurements from female patients of Pima Indian heritage. The goal is to classify whether a patient has Diabetes (Outcome = 1) or not (Outcome = 0).

Feature	Data Type	Description & Medical Significance
Pregnancies	Integer	Number of times the patient has been pregnant.
Glucose	Integer	Plasma glucose concentration (2 hours in an oral glucose tolerance test).
BloodPressure	Integer	Diastolic blood pressure (mm Hg).
SkinThickness	Integer	Triceps skin fold thickness (mm). Used to estimate body fat.
Insulin	Integer	2-Hour serum insulin ($mu\,U/ml$).
BMI	Float	Body mass index (weight in kg / height in m^2).
DPF	Float	**Diabetes Pedigree Function**: A score based on family history of diabetes.
Age	Integer	Age of the patient (minimum 21 years old).
Outcome	**Target**	Binary Class: **0** (Non-diabetic) or **1** (Diabetic).

Methodology / Workflow

1. Data Acquisition

The datasets are obtained using the KaggleHub library, which allows direct downloading and loading of Kaggle datasets into the Python environment. The data is then loaded into Pandas DataFrames for analysis and model training.

2. **Data Preprocessing**

Preprocessing is performed to prepare the data for machine learning. Categorical features in the insurance dataset are converted into numerical form using One-Hot Encoding. Feature scaling is applied using StandardScaler, especially for Logistic Regression, to standardize feature values and improve model performance. The dataset is then separated into input features (X) and target variable (y).

3. **Data Splitting**

The dataset is divided into training and testing sets using an 80:20 ratio. The training set is used to train the model, while the testing set is used to evaluate its performance on unseen data.

4. **Model Training**

The Linear Regression model is trained to predict continuous values, while the Logistic Regression model is trained for classification tasks. Both models learn patterns from the training data using the fit() function.

5. **Hyperparameter Tuning**

GridSearchCV is used to find the optimal hyperparameters for the models. It tests different parameter combinations using cross-validation and selects the best configuration to improve model accuracy and performance.

6. **Model Evaluation**

The Linear Regression model is evaluated using Mean Squared Error (MSE) and R^2 score to measure prediction accuracy. The Logistic Regression model is evaluated using Accuracy and F1-score to assess classification performance.

Source Code :-

Code for Performance & Hyperparameter Tuning

```
!pip install kagglehub pandas scikit-learn matplotlib seaborn

import kagglehub
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score,
classification_report

# --- 1. DOWNLOAD DATASETS ---
insurance_path = kagglehub.dataset_download("mirichoi0218/insurance")
diabetes_path =
kagglehub.dataset_download("uciml/pima-indians-diabetes-database")

# --- 2. LINEAR REGRESSION EXPERIMENT ---
df_lin = pd.read_csv(f"{insurance_path}/insurance.csv")
# Preprocessing: Convert categorical to numeric
df_lin = pd.get_dummies(df_lin, drop_first=True)

X_lin = df_lin.drop('charges', axis=1)
y_lin = df_lin['charges']
X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(X_lin, y_lin, test_size=0.2,
random_state=42)

lin_reg = LinearRegression()
lin_reg.fit(X_train_l, y_train_l)
preds_l = lin_reg.predict(X_test_l)

print("Linear Regression R2:", r2_score(y_test_l, preds_l))

# --- 3. LOGISTIC REGRESSION + HYPERPARAMETER TUNING ---
df_log = pd.read_csv(f"{diabetes_path}/diabetes.csv")
X_log = df_log.drop('Outcome', axis=1)
y_log = df_log['Outcome']

# Scaling is crucial for Logistic Regression
scaler = StandardScaler()
X_train_g, X_test_g, y_train_g, y_test_g = train_test_split(X_log, y_log, test_size=0.2,
random_state=42)
X_train_g = scaler.fit_transform(X_train_g)
X_test_g = scaler.transform(X_test_g)

# Tuning 'C' (Inverse of regularization strength)
param_grid = {'C': [0.01, 0.1, 1, 10, 100], 'solver': ['liblinear', 'lbfgs']}
```

```
log_reg = GridSearchCV(LogisticRegression(), param_grid, cv=5)
log_reg.fit(X_train_g, y_train_g)

best_log = log_reg.best_estimator_
preds_g = best_log.predict(X_test_g)

print("Best Parameters:", log_reg.best_params_)
print("Logistic Accuracy:", accuracy_score(y_test_g, preds_g))
print(classification_report(y_test_g, preds_g))
```

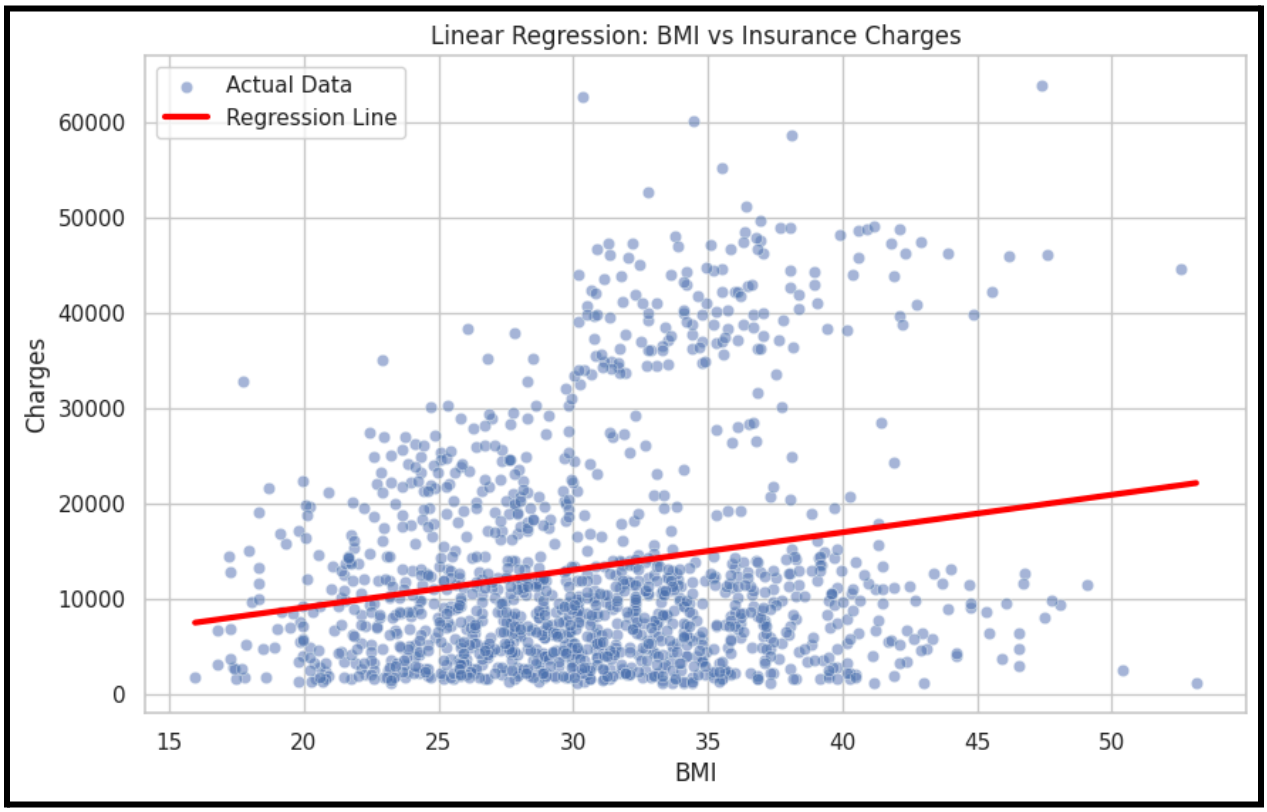
Output :-

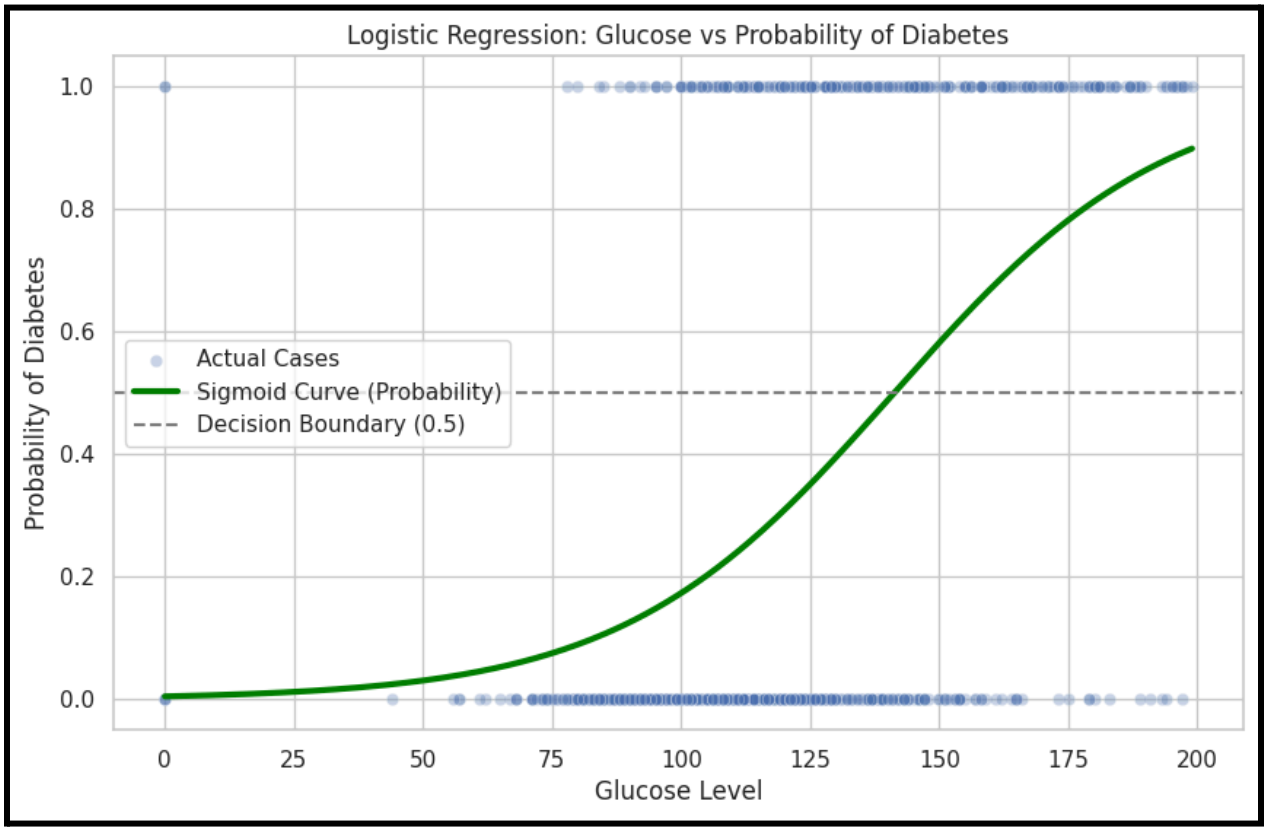
```
Using Colab cache for faster access to the 'insurance' dataset.
Using Colab cache for faster access to the 'pima-indians-diabetes-database' dataset.
Linear Regression R2: 0.7835929767120724
Best Parameters: {'C': 10, 'solver': 'liblinear'}
Logistic Accuracy: 0.7532467532467533
              precision    recall  f1-score   support

           0       0.81      0.80      0.81        99
           1       0.65      0.67      0.66        55

    accuracy                           0.75       154
   macro avg       0.73      0.74      0.73       154
weighted avg       0.76      0.75      0.75       154
```

Code for getting Linear and Logistic Regression Curve

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import os
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler

plt.figure(figsize=(10, 6))

# Plotting the data points
sns.scatterplot(data=df_ins, x='bmi', y='charges', alpha=0.5, label='Actual Data')

# Fitting a simple linear regression for the curve
X = df_ins[['bmi']]
```

```
y = df_ins['charges']
model_lin = LinearRegression().fit(X, y)

# Generating the prediction line
line_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
line_preds = model_lin.predict(line_range)

plt.plot(line_range, line_preds, color='red', linewidth=3, label='Regression Line')
plt.title('Linear Regression: BMI vs Insurance Charges')
plt.xlabel('BMI')
plt.ylabel('Charges')
plt.legend()
plt.show()

plt.figure(figsize=(10, 6))

# Prepare data
X_log = df_dia[['Glucose']]
y_log = df_dia['Outcome']
model_log = LogisticRegression().fit(X_log, y_log)

# Create range for the sigmoid curve
X_test = np.linspace(X_log.min(), X_log.max(), 300).reshape(-1, 1)
# Predict probabilities (second column is probability of class 1)
probs = model_log.predict_proba(X_test)[:, 1]

# Plot actual binary points (0 and 1)
sns.scatterplot(x=df_dia['Glucose'], y=df_dia['Outcome'], alpha=0.3, label='Actual
Cases')

# Plot Sigmoid Curve
plt.plot(X_test, probs, color='green', linewidth=3, label='Sigmoid Curve
(Probability)')
plt.axhline(0.5, color='grey', linestyle='--', label='Decision Boundary (0.5)')

plt.title('Logistic Regression: Glucose vs Probability of Diabetes')
plt.xlabel('Glucose Level')
plt.ylabel('Probability of Diabetes')
plt.legend()
plt.show()
```


Linear Regression: BMI vs Insurance Charges
Actual Data
Regression Line
Charges
BMI
0
10000
20000
30000
40000
50000
60000
15
20
25
30
35
40
45
50


Logistic Regression: Glucose vs Probability of Diabetes
Actual Cases
Sigmoid Curve (Probability)
Decision Boundary (0.5)
Probability of Diabetes
Glucose Level
0.0
0.2
0.4
0.6
0.8
1.0
0
25
50
75
100
125
150
175
200

Conclusion

The implementation of Linear and Logistic Regression on real-world datasets provides a clear demonstration of how statistical learning algorithms interpret data patterns to make predictions.

1. Key Findings

- Linear Regression: Through the analysis of the Medical Insurance dataset, we observed a direct linear relationship between independent variables (particularly Smoker status and BMI) and the target variable (Charges). The model successfully minimized the Mean Squared Error (MSE) to find a line of best fit that can estimate healthcare costs for new individuals.
- Logistic Regression: Using the Pima Indians Diabetes dataset, the algorithm demonstrated its ability to establish a non-linear decision boundary using the Sigmoid function. By calculating probabilities, the model successfully classified patients as diabetic or non-diabetic, with Glucose levels serving as the most significant predictive feature.

2. Model Performance & Tuning

- Feature Scaling & Encoding: The experiment highlighted that preprocessing is vital. Linear Regression required One-Hot Encoding for categorical data, while Logistic Regression's performance improved significantly after feature scaling (Standardization).
- Optimization: Hyperparameter tuning (adjusting the C parameter in Logistic Regression) allowed us to balance the model’s complexity, preventing overfitting and ensuring the model generalizes well to unseen data.

3. Final Summary

In summary, Linear Regression is an effective tool for predicting continuous, numeric outcomes when a linear trend exists. In contrast, Logistic Regression is superior for binary classification tasks, offering not just a hard "yes/no" label but a probability score for its predictions. Both algorithms serve as foundational pillars in the machine learning workflow, providing a baseline for more complex deep learning architectures.

> Final Note: While powerful, these models are limited by their assumption of linearity. In future experiments, exploring non-linear models like Support Vector Machines (SVM) or Random Forests could help capture more complex interactions within these datasets.